1. What are the most common reasons for missing data in ETL pipelines?

➔ In an ETL pipeline, missing data (often represented as `NaN` or `NULL`) rarely happens by accident. It usually stems from specific issues in the data lifecycle:

- **Data Entry Errors:** Manual input by users often leads to skipped fields or typos that the system cannot recognize.
- **System Integration Failures:** During the **Extract** phase, technical glitches or mismatched schemas between the source and target can cause data loss.
- **Non-Mandatory Fields:** In many applications, certain information (like "Income" or "Middle Name") is optional, meaning users simply choose not to provide it.
- **Sensor or Hardware Malfunctions:** In IoT or real-time streaming ETL, hardware failures can lead to gaps in time-series data.
- **Privacy and Security:** Certain data might be stripped out for compliance (like GDPR) before it reaches the ETL pipeline.

2. Why is blindly deleting rows with missing values considered a bad practice in ETL?

➔ Blindly deleting rows, also known as listwise deletion, is often discouraged in professional ETL workflows for several critical reasons:

- **Loss of Valuable Information**: Even if one column (like "Income") is missing, other columns in that row (like "City" or "Monthly_Sales") may contain vital data for different analyses.
- **Introduction of Bias**: If data is not missing randomly (e.g., lower-income individuals choose not to report it), deleting those rows skews the remaining dataset toward a specific demographic, leading to inaccurate business insights.
- **Reduced Sample Size**: For smaller datasets, deleting even a few rows can significantly reduce the statistical power of your models.
- **Loss of Context**: Sometimes the "missingness" is an insight in itself—for instance, if "Region" is missing, it might indicate a failure in a specific branch's data entry system.

3. Explain the difference between:
   - Listwise deletion
   - Column deletion

   Also mention one scenario where each is appropriate.

➔ Both methods involve removing data, but they differ in the "axis" of deletion.

Listwise Deletion (Row-wise)

- **Definition**: Removing the entire record (row) if any of its values are missing.

- **Scenario**: Appropriate when the missing value is a **Primary Key** or a critical field (like "Customer_ID") without which the record is useless.
- **MySQL Example**: `DELETE FROM customers WHERE Customer_ID IS NULL;`

Column Deletion

- **Definition**: Removing an entire attribute (column) from the dataset if it has too many missing values.
- **Scenario**: Appropriate when a column has a very high percentage of missing data (e.g., 90% of "Middle Name" is empty) and provides no predictive value to the analysis.
- **MySQL Example**: `ALTER TABLE customers DROP COLUMN middle_name;`

4. Why is median imputation preferred over mean imputation for skewed data such as income?

➔ In ETL, choosing the right statistical measure for filling gaps is crucial for data quality.

- **Mean Imputation**: The average is highly sensitive to outliers. In a column like "Income," a few billionaires would inflate the mean, making the imputed values unrealistically high for the average customer.
- **Median Imputation**: The median is the middle value. It is **robust to outliers** and provides a more realistic "typical" value for skewed data.

5. What is forward fill and in what type of dataset is it most useful?

➔ Forward fill is a data imputation technique where a missing value is replaced by the most recent valid value preceding it in the dataset.

- **Logic:** It assumes that the state of the data remains constant until a new change is recorded.
- **Best Use Case:** This is most useful in **Time-Series datasets** or sequential logs, such as temperature readings, stock prices, or sensor data, where the last known value is the best estimate for the current missing one.

6. Why should flagging missing values be done before imputation in an ETL workflow?

➔ **Flagging** involves creating an auxiliary column (a "missing indicator") to mark which values were originally missing before they are filled in.

- **Preserving Metadata:** It allows the system to distinguish between a "real" zero (or mean) and an "imputed" one.
- **Model Accuracy:** Machine learning models can use the flag as a feature, as the fact that data is missing may itself be a predictor of a certain outcome.

- **Audit Trail:** It maintains transparency in the ETL process, allowing data scientists to evaluate how much the imputation is influencing the final analysis.

7. Consider a scenario where income is missing for many customers. How can this missingness itself provide business insights?

➔ Missing data isn't always a technical error; it can represent a specific behavior or status.
- **Privacy Concerns:** High missingness in income might indicate that customers do not trust the platform with sensitive financial data.
- **Target Demographics:** It might reveal that certain age groups or regions are more hesitant to provide financial details than others.
- **System Friction:** If income is missing specifically for mobile users, it might suggest the mobile UI makes that specific field hard to fill out.

8. **Listwise Deletion**
   Remove all rows where **Region** is missing.
   Tasks:
   1. Identify affected rows
   2. Show the dataset after deletion
   3. Mention how many records were lost

➔ In MySQL, we use the DELETE statement to remove rows where a specific column is NULL.

1. Identify affected rows: The query below identifies **Customer 105 (Amit Verma)** as the row with the missing Region.

```
SELECT * FROM customers WHERE Region IS NULL;
```

2. Execute deletion and show result:

```
DELETE FROM customers WHERE Region IS NULL;

SELECT * FROM customers;
```

3. Records lost: Running this removes **1 record** from the database

9. **Imputation**
   Handle missing values in **Monthly_Sales** using:
   - **Forward Fill**

   Tasks:
   1. Apply forward fill

2. Show before vs after values
3. Explain why forward fill is suitable here

➔ MySQL does not have a built-in "Forward Fill" function like Python's Pandas, so we use a session variable to carry the previous value forward.

1. Apply forward fill:

```
SET @val := 0;

SELECT

Customer_ID,

Name,

@val := COALESCE(Monthly_Sales, @val) AS Monthly_Sales_Filled

FROM customers

ORDER BY Customer_ID;
```

2. Before vs. After values:

- Anjali Rao (102): NaN → 12000 (carried from Rahul).
- Neha Singh (104): NaN → 15000 (carried from Suresh).
- Karan Shah (106): NaN → 18000 (carried from Amit).

3. Suitability: Forward fill is suitable here if the data is sorted chronologically or by a sequence where the "last known state" is the most accurate estimate for the missing gap.

10. Flagging Missing Data

Create a **flag column** for missing Income.
Tasks:
Create Income_Missing_Flag (0 = present, 1 = missing)
Show updated dataset
Count how many customers have missing income

➔ Flagging is done using a `CASE` statement to create a virtual column during the extraction or transformation phase.

1. Create Income_Missing_Flag:

```
SELECT
```

```
Customer_ID,

Name,

Income,

CASE

WHEN Income IS NULL THEN 1

ELSE 0

END AS Income_Missing_Flag

FROM customers;
```

2. Count missing customers:

```
SELECT COUNT(*) AS Total_Missing_Income

FROM customers

WHERE Income IS NULL;
```

3. Result: The count will return **3**, as Anjali, Neha, and Pooja have missing income values in the dataset.